

KAIROX: Adaptive GPU-CPU Hybrid LLM Inference via Online Neuron Balancing

自适应 GPU-CPU 混合大模型推理

Yapeng Jiang, Minghao Gan, Zicong Hong, Wuhui Chen, Junyuan Liang, Yue Yu, Meng Guo, Zibin Zheng

Presenter: 徐凌飞

2026.09.07

Contents

1 Background & Motivation

2 Design

3 Evaluation

4 Thinking

研究动机：本地部署受显存上限约束

- 数据主权与隐私合规
- 避免持续云端调用成本
- 消除网络往返带来的时延抖动

模型规模持续超过显存容量

13B FP16 权重 > 26 GB

RTX 4090 VRAM = 24 GB

显存缺口迫使系统将部分权重置于
CPU 内存

研究问题：模型无法完整驻留显存时，如何利用 CPU-GPU 协同而不让 CPU 成为关键路径瓶颈？

FFN 是异构推理的主导存储与计算开销

FFN 参数占比



≈ 70%

Prefill 计算占比



≈ 60%

Decode 计算占比

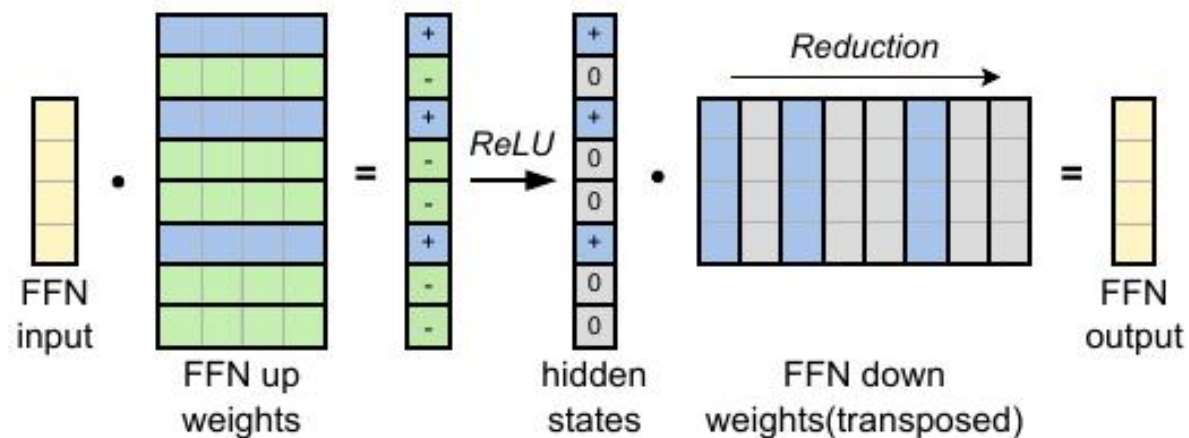


≈ 80%

优化 FFN 相当于直接作用于本地推理的主要存储与计算开销。

KAIROX 选择在 FFN 神经元粒度上做异构放置与动态迁移。

激活稀疏性提供细粒度异构放置机会



输入依赖

激活集合随上下文变化，不能在推理前完全确定。

长尾分布

少量神经元高频激活，多数神经元仅偶发参与。

可预测性

相邻层状态可支持对下一层激活的一层前瞻。

激活稀疏性使神经元级放置成为可能；接下来的关键是 GPU-CPU 边界能否适应真实输入。

Background & Motivation

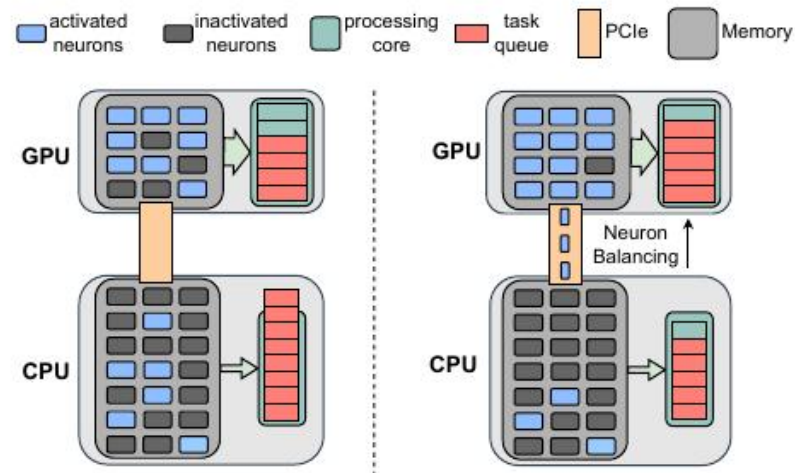
现有系统以静态切分换取可执行性，但依赖负载稳定假设

llama.cpp

以层为粒度切分 CPU/GPU；同步等待容易形成 GPU 空泡。

PowerInfer

按离线频率固定 hot/cold 神经元；隐含假设是激活模式稳定。

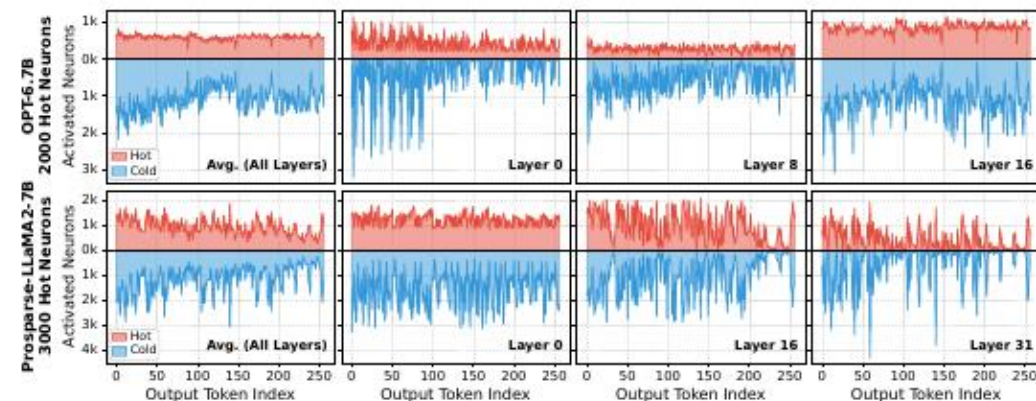
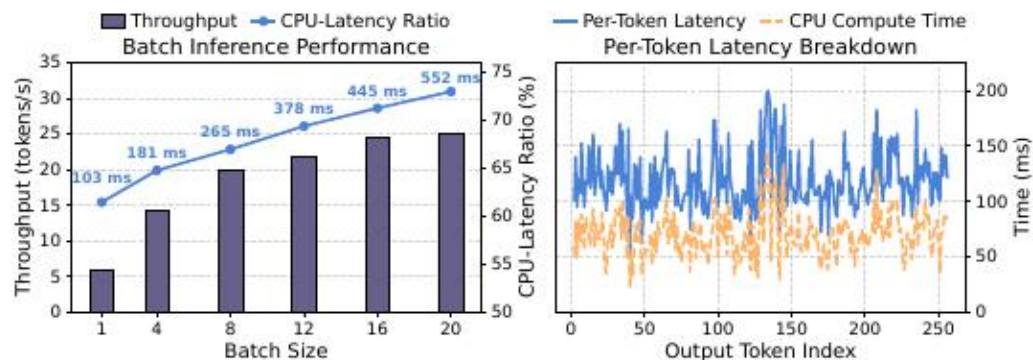


共同假设：运行时负载不会显著偏离预先确定的 **CPU-GPU placement**。

问题由此转化为：真实 **workload** 是否足够稳定，能让一次离线划分持续有效？

Background & Motivation

Batch 扩展与语义漂移都会把动态负载推向 CPU



Batch 扩展

batch 由 1 增至 20 时，CPU 延迟约由 103 ms 增至 552 ms；GPU 仍有并行余量。

语义漂移

生成过程中 cold 激活会连续突增，TPOT 可由约 50 ms 波动至接近 200 ms。

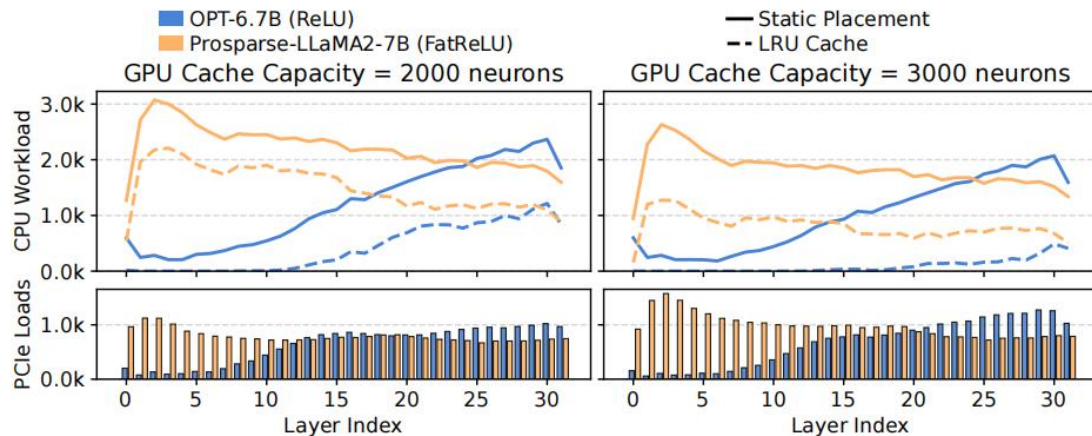
离线 profile 不能代表运行时；静态 hot/cold 划分会将动态负载错误地保留在 CPU。

Background & Motivation

在线重平衡可卸载 CPU 压力，但必须支付 PCIe 迁移代价

↓ CPU
迁移更多神经元到
GPU

↑ PCIe
更多权重跨设备搬运



挑战 1

迁移如何与计算重叠？

挑战 2

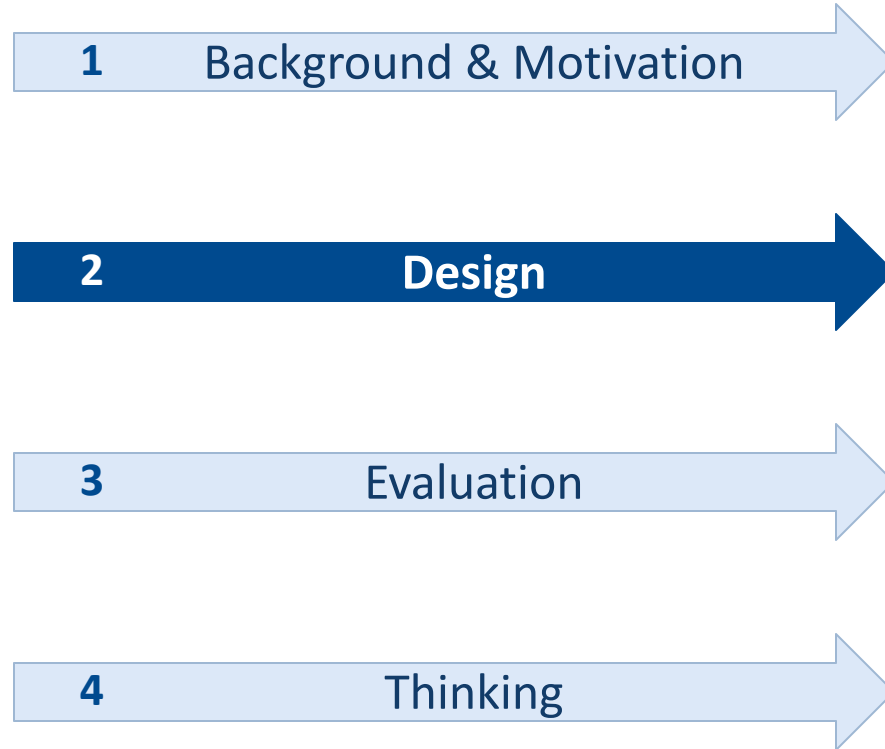
如何抑制瞬时热点迁移？

挑战 3

如何随瓶颈调节迁移强度？

KAIROX 分别以 Live Pipeline、TAM 与反馈控制应对三类系统问题。

Contents



KAIROX: 离线数据重排与在线自适应执行协同

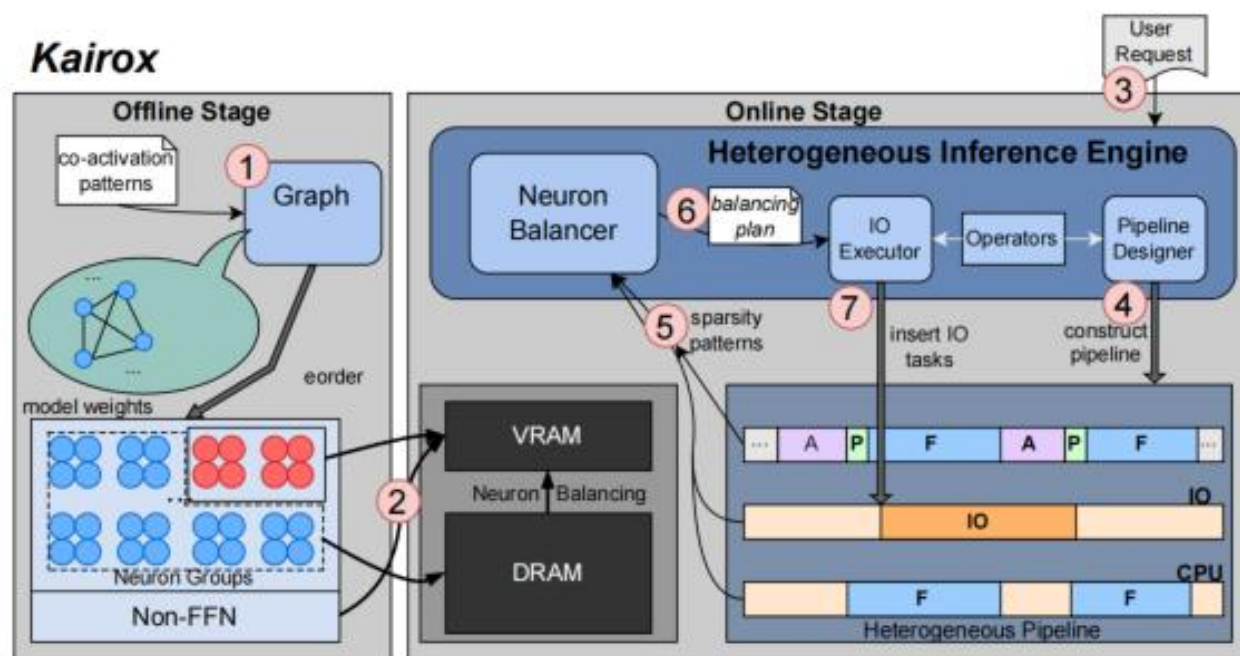


Figure 6: System overview of KAIROX.

① Live Pipeline

一层前瞻与异步 I/O 缩短数据重载的关键路径。

② TAM

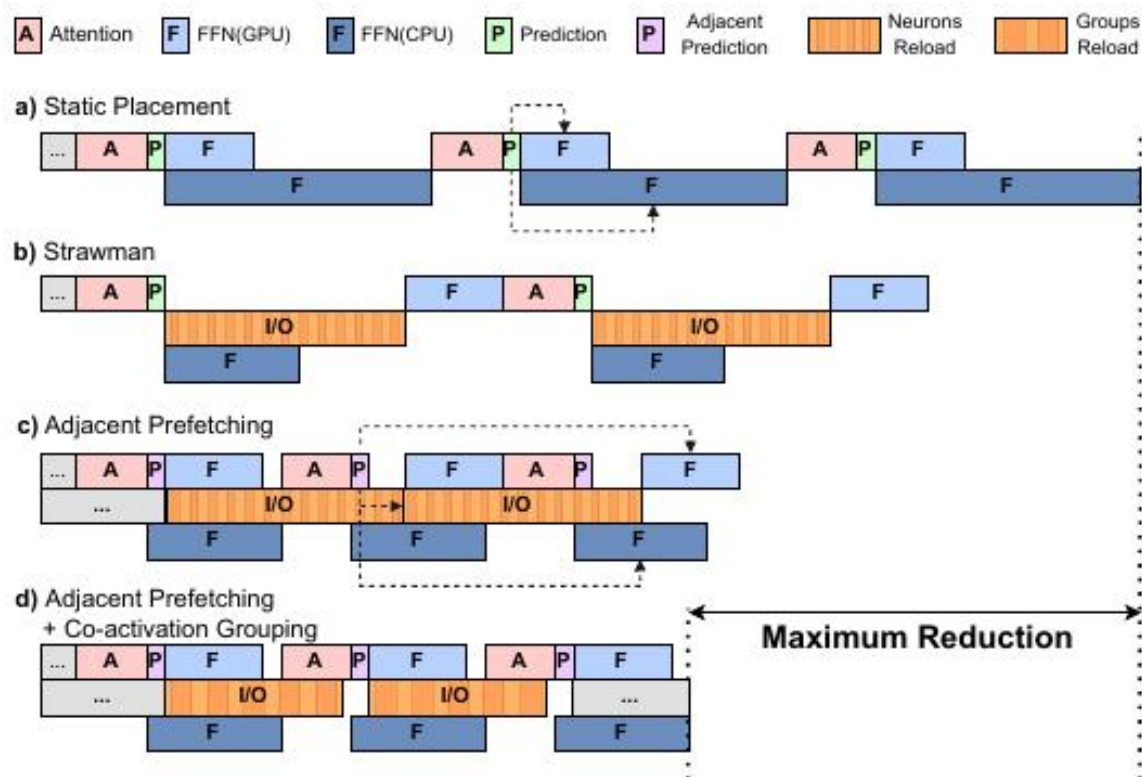
以时间动量估计缓存价值，抑制瞬态激活的迁移。

③ Adaptive Balancer

依据 CPU/I-O stall 反馈动态调节迁移强度。

离线阶段重排数据布局；在线阶段针对每个 token 的真实资源状态重新平衡。

Live Pipeline: 用一层前瞻消除重载关键路径



静态执行

Prediction $\rightarrow$ I/O $\rightarrow$ FFN
 串行依赖暴露 I/O 延迟

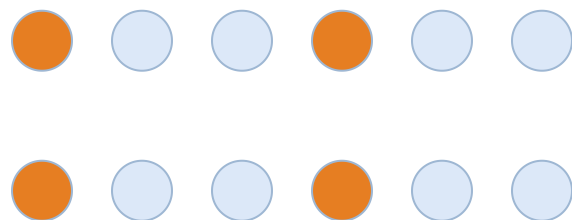
相邻层预测

利用第 i 层 Attention
 预测第 $i+1$ 层 FFN 激活
 使预取与当前计算并行

相邻隐藏状态余弦相似度约为 98%，一层前瞻在预测准确性与可重叠窗口之间取得平衡。

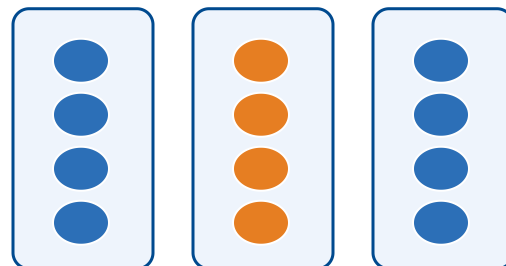
Live Pipeline: 协同激活分组提高 PCIe 有效带宽

单神经元迁移



碎片化 PCIe read
仅达到理论带宽的 18%~21%

METIS 分组与重排



高共激活神经元置于同一
连续组

组级迁移

10.2

PCIe 3.0 GB/s
(group size = 16)

20.9

PCIe 4.0 GB/s
(group size = 16)

Table 2: PCIe bandwidth (GB/s) under different hidden dimensions and neuron group sizes. The PCIe 3.0 and PCIe 4.0 results correspond to models with hidden sizes of 4096 (typical for 7B model) and 5120 (typical for 13B model), respectively.

GS	1	4	8	16	24	32	Theo.
PCIe 3.0	3.3	6.8	8.7	10.2	10.8	11.1	16
PCIe 4.0	5.8	13.7	17.8	20.9	22.3	22.8	32

组粒度增大可提高带宽利用率，但会增加 over-fetch；KAIROX 在二者之间选择折中。

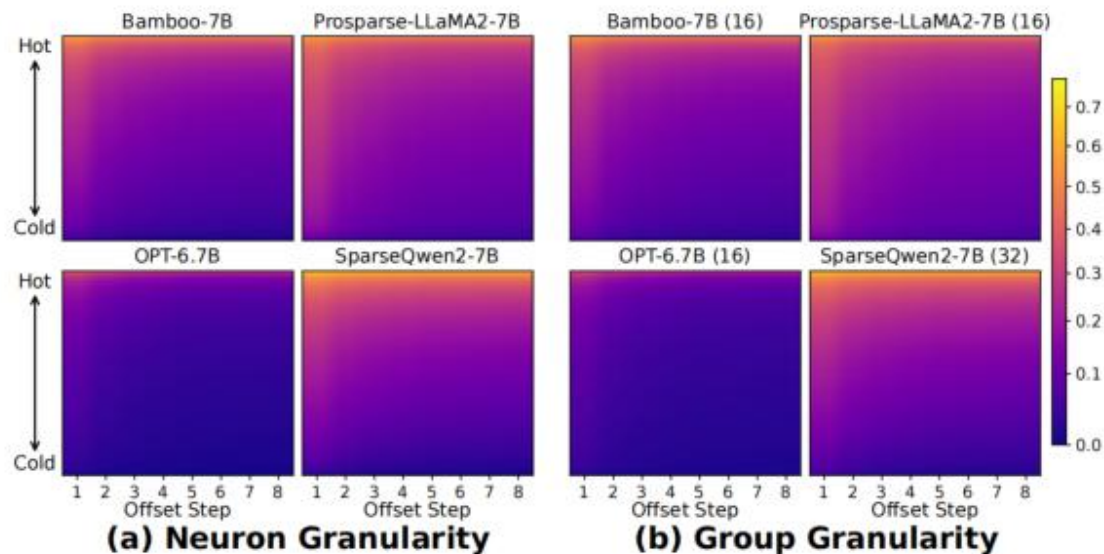
激活具有时间局部性，LRU 无法辨识持续价值

高频神经元

激活窗口更长，未来多个 token 仍可能持续使用。

低频神经元

底部 70% 的低频神经元衰减更快； $\delta > 2$ 时重激活概率常低于 0.2。



LRU 的局限 单次偶发激活会被提升至缓存顶部，进而挤出真正具有持续复用价值的神经元。

缓存策略应追踪神经元的持续价值，而非仅依据最近一次访问。

TAM(Temporal Activation Momentum): 以时间动量估计神经元的长期价值

$$S_t = \lambda \cdot S_{t-1} + (1 - \lambda) \cdot A_t$$

A_t : 当前激活强度 S_t : 累积动量分数 λ : 历史惯性

瞬态激活



短暂峰值被动量平滑，不急于迁移

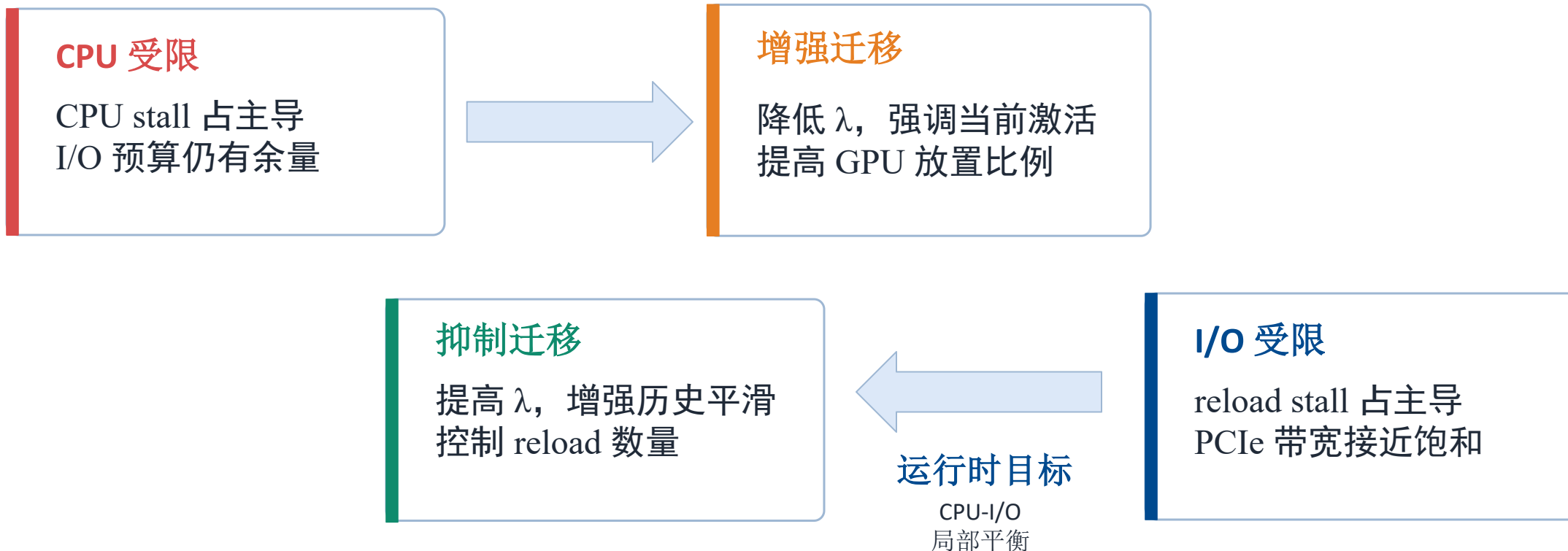
持续激活



持续激活累积高分，优先进入 GPU 缓存

TAM 决定迁移对象； λ 控制分数更新速度，因此可进一步作为反馈调节的控制变量。

Adaptive Neuron Balancer: 基于 stall 的闭环控制



KAIROX 不追求静态最优，而是持续追踪随输入和资源状态变化的动态平衡点。

实现：在 llama.cpp 上构建可执行的异构推理栈

5,700

新增 C++ / CUDA 代码行

400k+

离线 activation patterns

3

CUDA streams

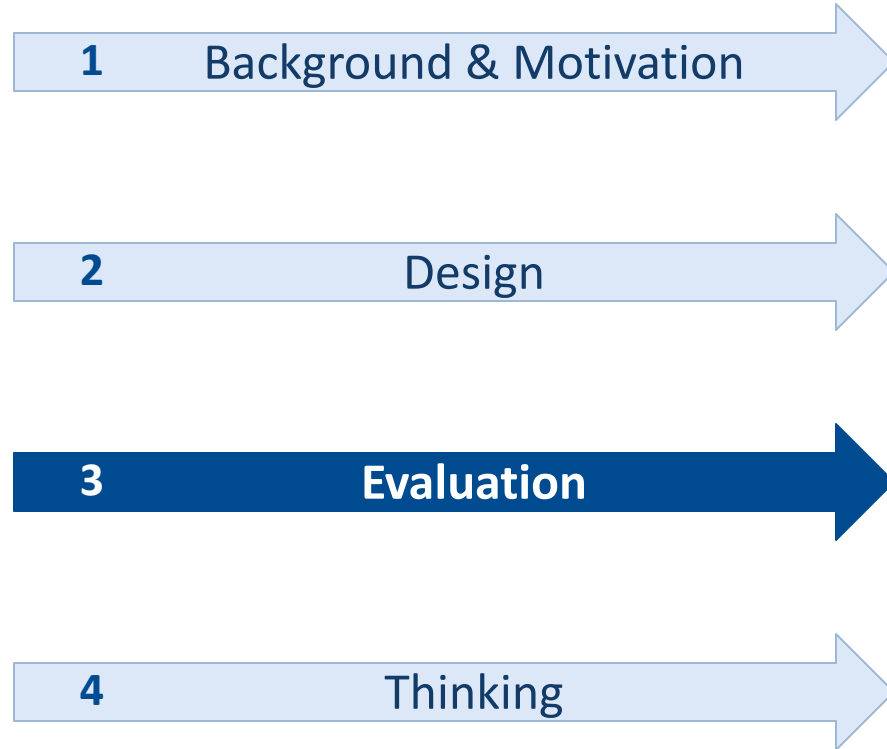
<1024

GPU-side group ranking
size

- 离线：在 C4 上训练相邻层预测器，并以 METIS 划分共激活图。
- 在线：以三个 CUDA stream 分离计算、关键 I/O 与 background reload。
- 内核：GPU 融合 TAM 与集合更新，CPU/GPU 使用定制稀疏内核。

系统贡献不仅是策略，还包括可执行异构 **DAG**、流优先级与定制稀疏内核。

Contents



实验设置：两档消费级平台、五类模型与四组基线

PC-Low

RTX 3080 Ti · 12 GB

Xeon E5-2680 v4 · 12 threads

64 GB DRAM · PCIe 3.0

PC-High

RTX 4090 · 24 GB

AMD EPYC 7542 · 16 threads

128 GB DRAM · PCIe 4.0

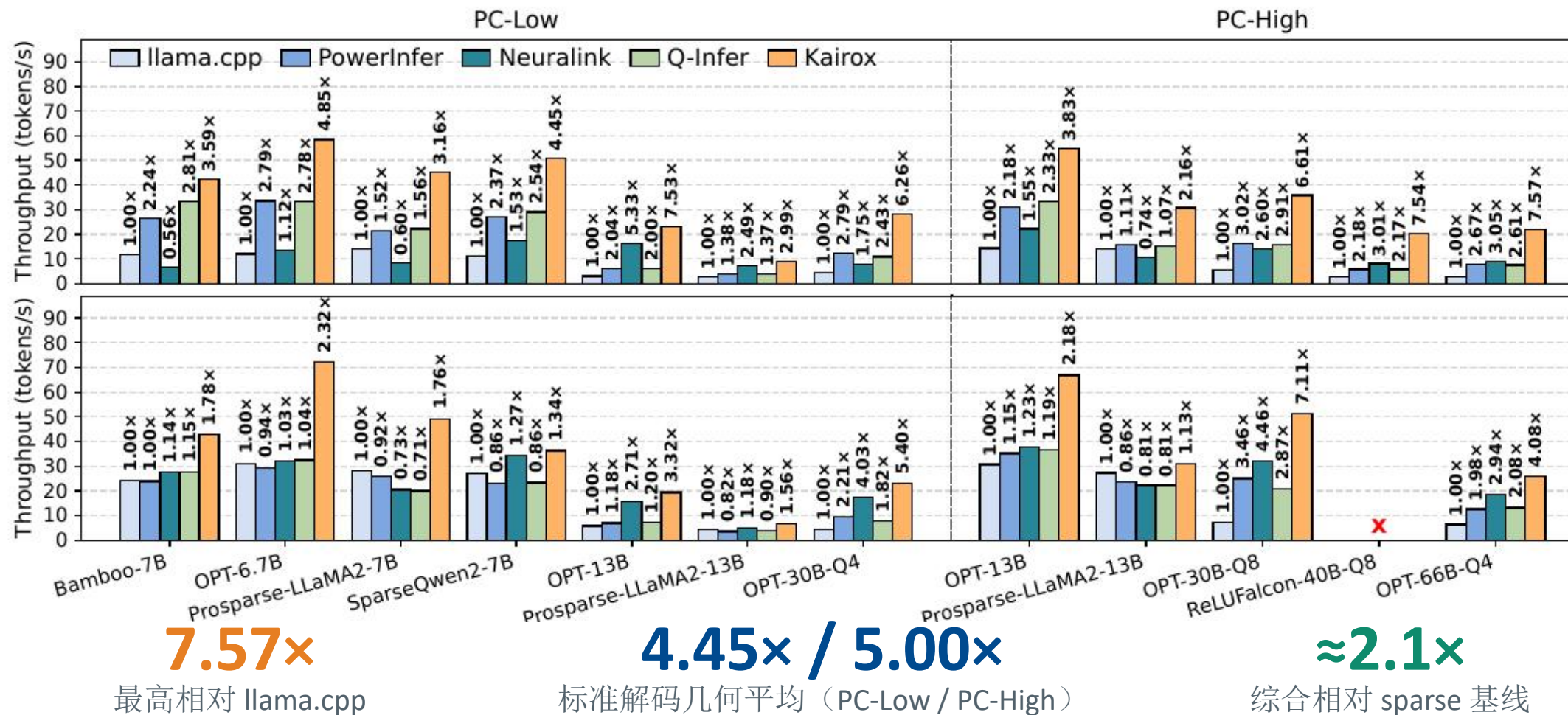
模型 Dense OPT: 6.7B-66B; Activation-sparse: Prosparse、Bamboo、SparseQwen2、ReLU Falcon

基线 llama.cpp（层级卸载）、PowerInfer（静态 hot/cold）与 Neuralink、Q-Infer

负载 ShareGPT; 标准解码 batch=1; output≤512; context≤1024

评测检验：在不同硬件、稀疏度与显存预算下，**KAIROX** 能否稳定缓解 CPU 与 I/O 瓶颈？

端到端评测：两档硬件上的吞吐优势具有一致性



性能增益来自对 GPU 利用率与 CPU-I/O 平衡的改善，而非增加显存或放宽精度。

反馈式 TAM 将 CPU 计算与 PCIe 传输稳定在更优区间

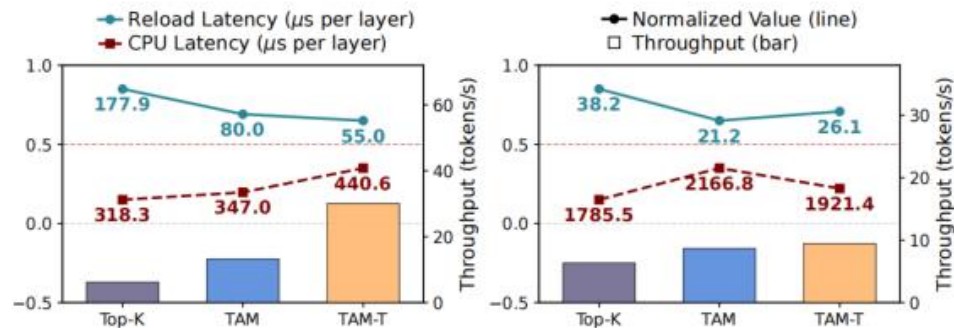


Figure 10: Comparison of CPU latency, I/O latency, and throughput under three reload policies. Left: Bamboo-7B on PC-Low. Right: Prospare-LLaMA2-13B on PC-Low. CPU latency and reload overhead are normalized.

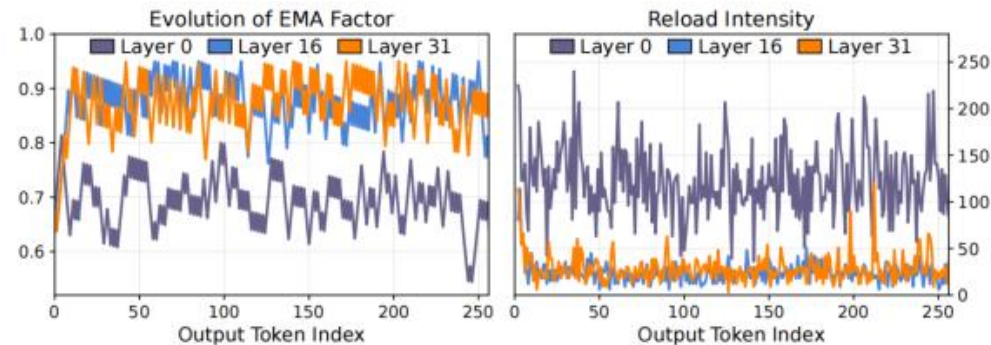


Figure 11: Evolution of the decay factor λ and the corresponding reload intensity, sampled from Bamboo-7B on PC-Low. Left: decay factor λ . Right: number of reload I/O tasks per step.

Top-K

迁移大量瞬态激活；CPU 负载下降，但 reload stall 抵消收益。

TAM

过滤短暂激活，使 reload latency 降低约 1.8-2.2 $\times$ 。

TAM-T

继续依据 stall 调节 λ ，自动匹配当前资源瓶颈。

TAM 决定迁移对象；feedback tuning 决定迁移强度。

消融结果：动态重平衡构成主要性能增益

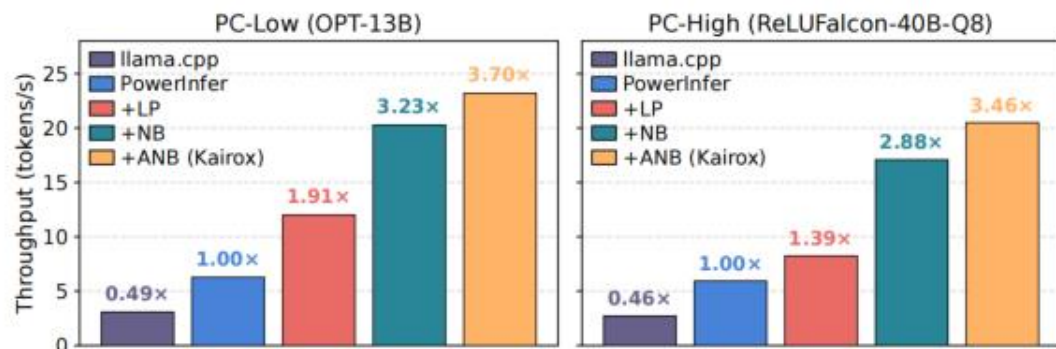


Figure 15: Contribution of each component in KAIROX, shown as throughput across incremental system variants.

1

PowerInfer

静态稀疏推理基线

2

+ LP(Live Pipeline)

重叠计算与通信：1.91x / 1.39x

3

+ NB(Neuron Balancing)

动态迁移缓解 CPU 瓶颈：3.23x / 2.88x

4

+ ANB(Adaptive Neuron Balancer)

闭环控制达到峰值：3.70x / 3.46x

Pipeline overlap 是基础；运行时重平衡神经元放置才释放主要异构计算潜力。

精度与开销：预测器高召回，系统属于近似稀疏推理

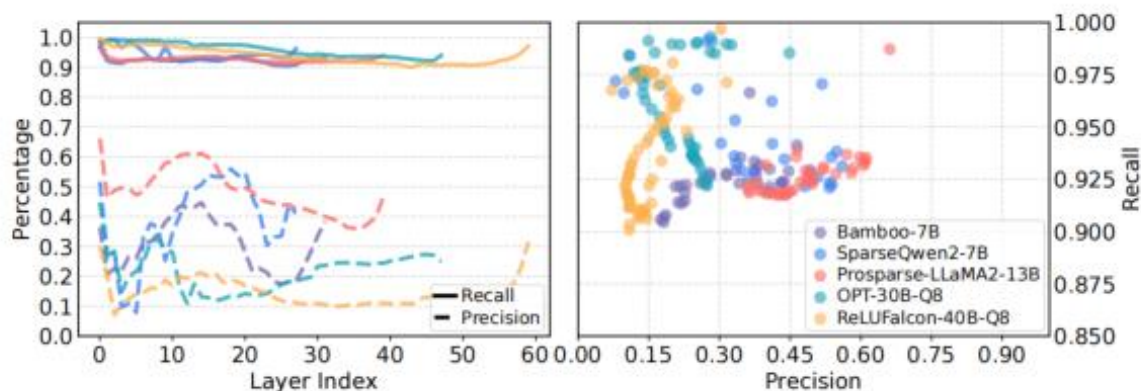


Figure 16: Predictor recall and precision across representative sparse models. Left: recall and precision across layers. Right: the corresponding precision–recall scatter plot for each predictor.

90-99%

预测器召回率

1-2%

预测器端到端开销

<0.5%

论文报告的平均任务精度下降

精度结果的解释

高 recall 减少漏加载；false positives 以额外传输换取覆盖。

平均精度下降较小，但个别任务仍有波动；因此应称为“近似稀疏推理”，而非严格无损。

性能收益依赖可预测的激活稀疏性；报告时应同时交代精度口径与适用范围。

外推边界：当前证据的适用范围与未解问题

① 场景边界

仅验证两台 NVIDIA 独显 PC；标准设置为 batch=1、context≤1024。

② 模型边界

低稀疏模型、MoE 路由与 dense MLP 的适配仍不完整。

③ 系统边界

未覆盖 NPU、统一内存 SoC、多 GPU 及 CXL/NVLink 等互连。

④ 离线成本

需采集激活、训练预测器、构建图划分；分布漂移时需重新验证。

评价结论 论文的关键价值是“基于运行时反馈动态改变放置”的系统方法，而非某个固定分配比例。

证据支持消费级离散 GPU-CPU 的动态放置；硬件、阶段与质量边界构成下一步扩展空间。

对异构设备项目的启示：多设备、分层粒度与质量约束



阶段感知

Prefill 偏 compute-bound; Decode 偏 memory-bound, 应采用差异化放置策略。

质量感知

将预测不确定性与精度预算纳入反馈控制, 而非仅观察 CPU / I-O stall。

可进一步研究：如何让 **placement policy** 同时感知阶段、设备、互连与精度风险？

Q & A

Adaptive placement for heterogeneous LLM inference

Presenter: 徐凌飞 · 2026.09.07